GitLab CI/CD uses a configuration file called `.gitlab-ci.yml` to define the pipeline structure, jobs, and stages. Below is a comprehensive list of commonly used GitLab CI/CD parameters, their definitions, and descriptions:

Top-Level Parameters

1. **stages**:
   - **Description**: Defines the stages of the pipeline.
   - **Example**:
     ```yaml
     stages:
       - build
       - test
       - deploy
     ```

2. **default**:
   - **Description**: Defines default parameters for all jobs.
   - **Example**:
     ```yaml
     default:
       image: ruby:2.7
     ```

3. **include**:
   - **Description**: Includes external YAML files.
   - **Example**:
     ```yaml
     include:
       - local: 'path/to/local.yml'
       - project: 'group/project-name'
     ```

```
    ref: 'master'
    file: '/templates/.gitlab-ci-template.yml'
```

4. **variables**:
   - **Description**: Defines global environment variables.
   - **Example**:
     ```yaml
     variables:
       DATABASE_URL: "postgres://postgres@postgres/my_database"
     ```

Job-Level Parameters

1. **script**:
   - **Description**: The commands to execute.
   - **Example**:
     ```yaml
     build-job:
       script:
         - echo "Building..."
     ```

2. **stage**:
   - **Description**: Specifies the stage for the job.
   - **Example**:
     ```yaml
     test-job:
       stage: test
       script:
```

```
    - echo "Testing..."
```

3. **image**:
   - **Description**: Specifies the Docker image to use.
   - **Example**:
     ```yaml
     build-job:
       image: node:14
       script:
         - npm install
         - npm run build
     ```

4. **tags**:
   - **Description**: Tags to select specific runners.
   - **Example**:
     ```yaml
     deploy-job:
       tags:
         - production
       script:
         - echo "Deploying..."
     ```

5. **only**:
   - **Description**: Specifies when to run jobs.
   - **Example**:
     ```yaml
     test-job:
     ```

```
    only:
      - master
```

6. **except**:
   - **Description**: Specifies when not to run jobs.
   - **Example**:
   ```yaml
   deploy-job:
     except:
       - branches
   ```

7. **allow_failure**:
   - **Description**: Allows jobs to fail without failing the pipeline.
   - **Example**:
   ```yaml
   lint-job:
     script:
       - eslint .
     allow_failure: true
   ```

8. **retry**:
   - **Description**: Specifies retry policy for jobs.
   - **Example**:
   ```yaml
   unstable-job:
     script:
       - ./run-unstable-job.sh
```

```
    retry:
      max: 2
      when: runner_system_failure
```

9. **artifacts**:
   - **Description**: Specifies artifacts to be stored after job completion.
   - **Example**:
   ```yaml
   build-job:
     script:
       - make build
     artifacts:
       paths:
         - build/
   ```

10. **cache**:
   - **Description**: Specifies files and directories to cache between jobs.
   - **Example**:
   ```yaml
   build-job:
     script:
       - npm install
     cache:
       paths:
         - node_modules/
   ```

11. **dependencies**:

- **Description**: Specifies job dependencies.
- **Example**:

  ```yaml
  test-job:
    script:
      - npm test
    dependencies:
      - build-job
  ```

12. **before_script**:
    - **Description**: Commands to run before each job.
    - **Example**:

      ```yaml
      before_script:
        - echo "This runs before each job"
      ```

13. **after_script**:
    - **Description**: Commands to run after each job.
    - **Example**:

      ```yaml
      after_script:
        - echo "This runs after each job"
      ```

14. **services**:
    - **Description**: Docker images to link to the job.
    - **Example**:

      ```yaml
      ```

```
test-job:
  services:
    - postgres:latest
  script:
    - npm test
```

15. **interruptible**:
   - **Description**: Allows jobs to be interrupted if a newer pipeline starts.
   - **Example**:
     ```yaml
     long-running-job:
       script:
         - sleep 3600
       interruptible: true
     ```

16. **timeout**:
   - **Description**: Maximum time a job can run.
   - **Example**:
     ```yaml
     long-running-job:
       script:
         - sleep 3600
       timeout: 30m
     ```

17. **parallel**:
   - **Description**: Specifies parallelism for jobs.
   - **Example**:

```yaml
parallel-job:
  script:
    - echo "Running in parallel"
  parallel:
    matrix:
      - VAR: 1
      - VAR: 2
```

Pipeline Configuration

1. **workflow**:
   - **Description**: Configures when pipelines are created.
   - **Example**:
     ```yaml
     workflow:
       rules:
         - if: '$CI_COMMIT_REF_NAME == "master"'
         - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
     ```

2. **include**:
   - **Description**: Includes other YAML files.
   - **Example**:
     ```yaml
     include:
       - template: Auto-DevOps.gitlab-ci.yml
       - local: 'my-custom.yml'
     ```

3. **extends**:
   - **Description**: Extends configuration from other jobs or templates.
   - **Example**:
     ```yaml
     .base-job:
       script:
         - echo "This is a base job"

     my-job:
       extends: .base-job
       script:
         - echo "This job extends the base job"
     ```

Environment and Deployment

1. **environment**:
   - **Description**: Defines environment-specific settings.
   - **Example**:
     ```yaml
     deploy-job:
       script:
         - echo "Deploying..."
       environment:
         name: production
         url: http://production.example.com
     ```

2. **when**:

- **Description**: Specifies when to run the job (e.g., on_success, on_failure, always).
- **Example**:

  ```yaml
  deploy-job:
    script:
      - echo "Deploying..."
    when: manual
  ```

3. **release**:
   - **Description**: Creates a GitLab Release.
   - **Example**:

     ```yaml
     release-job:
       stage: deploy
       script:
         - echo "Creating a release..."
       release:
         name: 'Release $CI_COMMIT_TAG'
         description: 'Release description'
         tag_name: '$CI_COMMIT_TAG'
         ref: '$CI_COMMIT_SHA'
     ```

This list covers the most commonly used parameters and settings for configuring GitLab CI/CD pipelines. For more detailed information and advanced configurations, refer to the [GitLab CI/CD documentation](https://docs.gitlab.com/ee/ci/yaml/).